Towards AI Online Course

Table of Contents

1Course Introduction and Understanding LLM(s)....4
1.1Why Prompt Engineering, RAG, Tools and Fine Tuning?....4
1.1.1Prompt Engineering....4
1.1.1.1Chain of Thought prompting 4
1.1.1.2Few Shot Prompting = Shot-Based Prompting....4
1.1.1.3RAG....4
1.1.1.4Fine Tuning....4
1.1.1.5Agents....4
1.2Understanding LLM(s) without math or Code....4
1.3Understanding LLMs Part 2: New Kind of Intelligence, New Kind of Stupidity 5
1.3.1Reducing Shortcomings With RAG, Prompting, Fine-Tuning and Tools....6
2Part 1: Building our RAG AI Tutor Using LLMs....6
2.1Introduction to LLM(s) and how to use via API....6
2.2To Use a LLM or not to use it?....6
2.3Limitations and Weaknesses of LLMs....6
2.4Choosing your LLM: Metrics, Benchmarks....6
2.5What is Prompting? Talking with AI Models....7
2.6Prompting 101....7
2.6.1Zero shot prompting....7
2.6.2In-Context learning and few shot prompting....7
2.6.3Role Prompting....7
2.6.4Chain Prompting....7
2.6.5Chain-of-Thought Prompting (CoT)....7
2.6.6Tips for Effective Prompt Engineering....7
2.6.7o1 and “Reasoning model” prompting....8
2.7What is Prompt Injection? Can you hack a prompt?....8
2.8Evaluating and Iterating Prompts....8
3Part 1: Building our RAG AI Tutor – RAG Basics....8
3.1What is RAG?....8
3.2RAG 101....9
3.2.1How does RAG work?....9
3.2.2Cache augmented Generation (CAG)....9
3.2.3Building a Basic RAG pipeline from scratch....9
3.2.3.1Choosing the best temperature parameter 10
4Part 1: Building our RAG AI Tutor – RAG with LlamaIndex 10
4.1Basic RAG with LlamaIndex 10
4.2How Vector DB(s) work and when to use one....10
4.3Using a Vector Database....11
4.3.1RAG Evaluation Techniques....11
4.3.1.1Retrieval metrics....12
4.3.1.2Generation Metrics....12
4.3.1.3Evaluation Dataset....12
4.4Evaluating your RAG pipeline....12
4.4.1The Golden Dataset....12
5Part 1: Building our RAG AI Tutor – Using LLM(s) and Embedding Models....13
5.1Privacy and Compliance concerns for LLM(s)....13
5.2Using Llama 3.1 70B on Together AI 13

5.3Multimodal LLM(s) and Their Role in RAG Pipelines....13
6Part 1: Building our RAG AI Tutor – Collecting, Filtering, Cleaning Data....14
6.1Scraping Data from Websites....14
7Part 1: Building our RAG AI: Advanced RAG....14
7.1Larger Context Window: More Chunks....14
7.2LLM as a Judge: Ranking Chunks....14
7.3Boost Your RAG Searches....14
7.4What is indexing? Indexing methods in Vector retrieval....15
7.4.1Hierarchical Navigable Small Worlds (HNSW)....15
8Part 1: Fine Tuning....15
8.1Optimizing the model: inference and fine tuning....15
8.2Fine Tuning 101....16
8.3Is Fine Tuning an embedding model worth?....16
8.4Fine tuning Embedding Models with Llama Index....17
9Part 1: Expanded RAG Toolkit....17
9.1Non technical skills for Effective LLM Development....17
9.1.1Business, Product and Entrepreneurial Skills....17
9.1.1.1Considering business strategy and economics. 17
9.1.1.2Communication and Management Skills 17
9.1.2New Learning and Development Design Philosophies....17
9.1.2.1Agile Learning and Keeping Up To Date....17
9.1.2.2System Design Philosophy and Quality Assurance....17
9.2Adding Question Validation and Routing....18
9.2.1Question routing....18
9.2.2Question validation....18
9.3An Introduction to Reasoning Language Models 18
9.4Enabling Conversational Memory in LLM(s)....18
9.5Adding Speech With GPT-4o Audio....19
9.6RAG vs Long Context 19
9.7Long Context LLM(s) & Context Caching vs RAG....19
9.7.1Google DeepMind's Michelangelo: Long Context Evaluations Beyond HayStacks via Latent Structure Queries....19
9.8Graph Machine Learning....19
9.8.1Graph Neural Networks (GNN(s))....20
9.8.2Graph Transformer-Based Model....20
9.8.3GraphRAG....20
10 Part 1: Building and Deploying the Final RAG Chatbot....21
10.1Building the final AI Tutor (link)....21
10.2How to make an API?....22
10.2.1Open API....22
10.2.2HTTP Methods....22
10.3Building and deploying a Gradio UI on Hugging Face Spaces....22
10.4Quick Tips for Improving LLM(s) once in production....22
11Part 2 Introduction: More Tools, Models & Frameworks for LLM Projects....24
11.1LLM Agents: LLM Pipelines and Tool Use for Complex tasks....24
12Part 3: LLM Ecosystem Economics; Choose the Right Niche and Business Strategy for your LLM Pipeline (9 Lessons)....24
12.1What Will Drive Enterprise Adoption of Generative AI?....24
12.2Who are the Generative AI winners?....24
12.3Impact of Large Language Models on Different Job Categories....24
12.4175 Project Ideas....25
12.4.1Technology and SW....25

12.5What Makes a Defensible Product Built With Foundation LLMs?....25
13Glossary and Terms....26
14Papers....26
14.1Constitutional AI: Harmlessness from AI Feedback (link)....26
15 Moreover....27
15.1Model Context Protocol (MCP)....27
15.1.1Blast radius of AI agents....27
15.1.2Creating an MCP server with Python....27
15.1.3Links....27
16Biography....28
17Links 28
18Tools....28
18.1AI Based Search Engines....28
18.2Companies 28
18.3Deployment....28
18.4Development Tools 28
18.5Models....28
18.6Text To Speech....28
18.7Training 28
18.7.1Prompt Engineering....28
18.7.2Post-Training 28
19Terms and acronyms....28

1 Course Introduction and Understanding LLM(s)

Course Home Page

1.1 Why Prompt Engineering, RAG, Tools and Fine Tuning?

https://academy.towardsai.net/courses/take/beginner-to-advanced-llm-dev/multimedia/59790904-why-prompt-engineering-rag-tools-and-fine-tuning

The key developer tools are:

- Prompt Engineering
- RAG
- Fine Tuning
- Custom UI

1.1.1 Prompt Engineering

Prompting (YouTube): how humans talk to the LLM. Prompts translate from Human Language to AI Language.

1.1.1.1 Chain of Thought prompting

(https://learnprompting.org/docs/intermediate/chain_of_thought) Incorporate the logical steps into the prompt. Provide a few examples, with the steps explained. Decompose the problem and guide with examples.

It works with models of > 100B parameters.

1.1.1.2 Few Shot Prompting = Shot-Based Prompting

https://learnprompting.org/docs/basics/few_shot The technique is called In-Context Learning. The model learns from the prompt itself. The number of shots are the number of examples contained in the prompt.

- Zero Shot: While zero-shot prompting can work well for simple tasks—especially ones that the model has likely encountered during training—it's often not enough for more complex tasks. The lack of examples leaves the model guessing, and the results can be unpredictable or incorrect.

1.1.1.3 RAG

(YouTube) The Model hallucinates because it lacks relevant context. RAG adds more knowledge or content to the prompt. We ground the model with knowledge we control. Disadvantage: the model knowledge is limited to what we provide, which is not as big as the internet. To create a RAG, the text is split into chunks of text eg 500 characters. From each chunk, we produce embeddings.

When the user has a question, the question is embedded. Then the relevant chunks are searched in the vector database. It is like searching for the relevant chapters in a textbook. Additional Questions:

- how to determine if the question was answered?
- If it is relevant or in your documentation.
- Fine tuning is an alternative to RAG
- When you update the knowledge base, the model is automatically updated, there is no need to change the model to acquire new knowledge.
- Note: if we detect that we do not have the knowledge to answer, we can acquire new knowledge automatically and add it to the knowledge base.

A critical factor is data freshness—in fast-moving fields such as finance or healthcare, outdated information can lead to inaccurate responses. Ensuring your dataset is continually updated is key to maintaining high-quality outputs in dynamic environments. Basic retrieval systems can also often fail to find the most useful information in your dataset, and even if they retrieve the most useful information - the final model might not use it in its answer. Making a successful RAG model is a complex process of trial and error and integrating more sophisticated techniques.

1.1.1.4 Fine Tuning

https://www.louisbouchard.ai/fine-tuning/

Fine tuning specializes a LLM into an expert in a specific task. Like from high school to university, to Phd. You start from general, into more specific knowledge. Losing general expertise, but gaining specific expertise. Fine tuning is needed when you need a domain specific expert.

However, fine-tuning comes with several challenges. One of the primary hurdles is building the right data source. To fine-tune effectively, you need high-quality, domain-specific data, which can be time-consuming and costly to gather and curate. Additionally, the process often demands substantial computational resources, especially when fine-tuning larger models, which can be prohibitive for some organizations. Another risk is that fine-tuning might cause the model to drift away from its pre-trained general-purpose knowledge, limiting its versatility in other areas.

1.1.1.5 Agents

There are blurred lines between advanced RAG, pipelines and agents.

1.2 Understanding LLM(s) without math or Code

AGI: intelligence traits are:

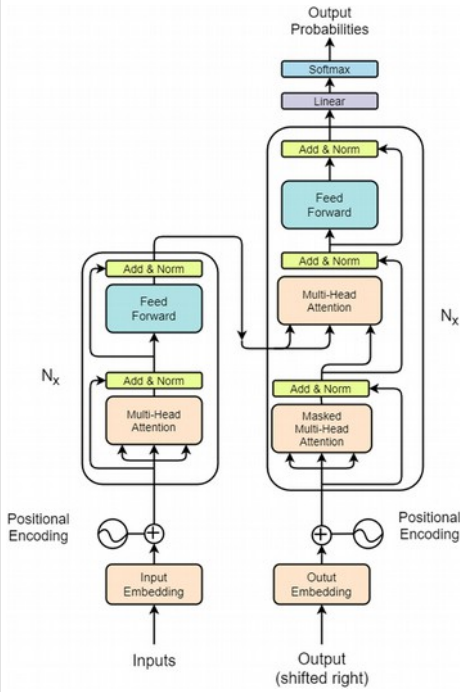
- reason, use strategy, solve puzzles, and make judgments under uncertainty
- represent knowledge, including common sense knowledge
- plan
- learn
- communicate in natural language
- if necessary, integrate these skills in completion of any given goal

LLM(s) are a lossy compression of the Internet. In large models, only one epoch is used.

Training and the transformer architecture. Words are turned into tokens. Each token is mapped to a vector in a high dimensional space called embedding. Space size about 2000 or 4000.

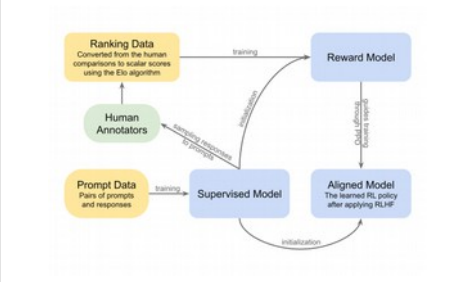
The reality is more complex than this example, but the point is that these embeddings capture semantic similarity through multiple dimensions, allowing the model to understand and generate contextually relevant text. It is also important to note that the meanings of these different dimensions are not set by humans - they are developed by the model itself during training.

Question: if embeddings are needed to train the model? How are embeddings trained?



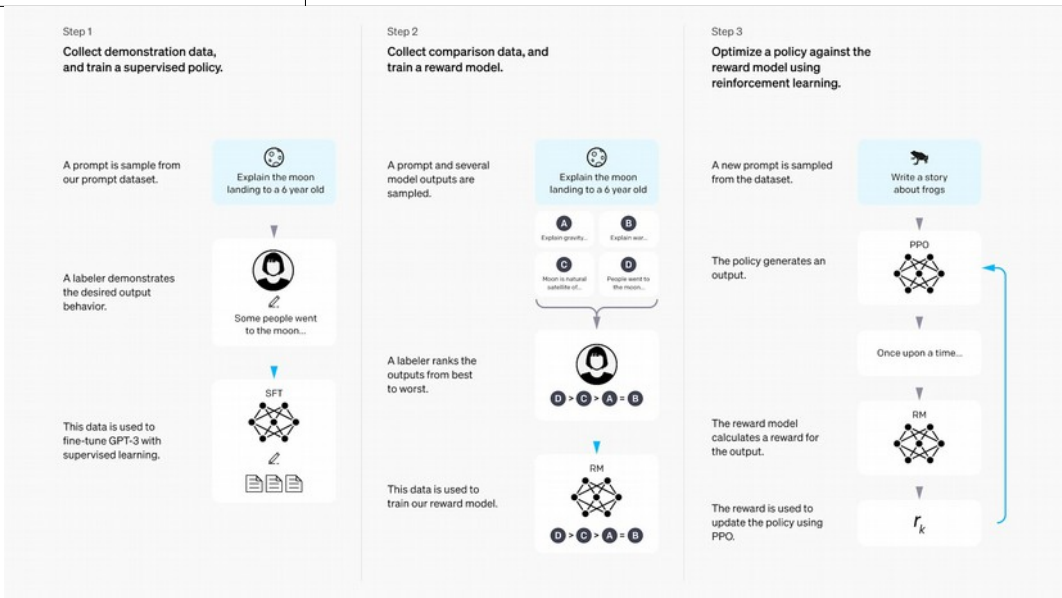
A key breakthrough of the transformer architecture (created in 2017) is the attention mechanism allowing all input tokens (say 500 words) to be processed within a single matrix in parallel in each layer - dramatically reducing the computational cost of training large models and making them well suited to GPUs. This paved the way for much larger models with more training data and more capability.

Mixture of Experts (MoE) Wikipedia Switch Transformers paper
https://poloclub.github.io/transformer-explainer/



RLHF Reinforcement Learning from Human Feedback. Post-training process: the model undergoes RLHF (Wikipedia) (Link to OpenAI article)

RLHF: Human Annotation trains a Reward Function (Reward model), with supervised training. The Reward Model is then used to align the Supervised Model. To apply RLHF, the task must be difficult but easy to judge.



1.3 Understanding LLMs Part 2: New Kind of Intelligence, New Kind of Stupidity

Is the model truly reasoning about the world and developing true intelligence just by lossy compression of its training data?
Maybe an LLM has more complex world models or reasoning systems developed and stored within its weights. Maybe it can just successfully search for and combine multiple useful "stupid patterns" or curves to create useful responses to a huge range of human requests. A key test is whether the model can develop emergent capabilities that can deliver successful responses outside of the distribution of its training data. Another key test is whether it still shows obvious reasoning failures.
One benchmark designed to test reasoning ability specifically - where results can't be achieved just by memorization, is the

Abstraction and Reasoning Corpus (ARC) benchmark. It presents diverse tasks that require pattern recognition, rule inference, and generalization. These tasks involve input and output grids that demand the model to infer transformation rules and apply them to new, unseen examples. Unlike typical AI benchmarks, ARC aims to make it impossible for models to succeed through memorization alone, thereby requiring true abstract reasoning and cognitive flexibility similar to human intelligence. The highest score on the ARC (Abstraction and Reasoning Challenge) is 49.0 (As of October 2024), achieved by the Minds AI team during a Kaggle competition. In the 2023 Arcathon, the same team scored 30.

Another common logic puzzle shows similar flaws: "*Sally has three brothers. Each brother has two sisters. How many sisters does Sally have?*" the model has been overfitting on similar-looking (yet different!) examples in its training data, and it uses these memorized facts for the next token prediction rather than the more general features. In these cases, a better prompt may have triggered it to find, use, and prioritize instead of the predictions from its more general features. Many things happen in LLMs during inference, but only one token is eventually sampled. Of course, this still points to flaws in their capabilities.

1.3.1 Reducing Shortcomings With RAG, Prompting, Fine-Tuning and Tools

Hugging Face: Introduction to State Space Models (Mamba on Wikipedia) https://github.com/microsoft/Samba

2 Part 1: Building our RAG AI Tutor Using LLMs

2.1 Introduction to LLM(s) and how to use via API

Research has demonstrated that repeatedly applying the same process across datasets comprising trillions of words and using billions of parameters unlocks the emergent abilities in LLMs. These abilities stem from the models' deep comprehension of language and are abilities that were not explicitly programmed into the models, such as reasoning and problem-solving.

2.2 To Use a LLM or not to use it?

Where LLM(s) make a difference: stuck on creative tasks (help with creativity), but double check LLM's work. They can be repetitive. Where LLM(s) do not hit the mark? They are not ready to make critical decisions.
Sometimes a Small Language Model (SLM) is enough: well trained small model, or distilled Llama 3 model can be enough It is also faster and less expensive.

2.3 Limitations and Weaknesses of LLMs

Hallucinations: LLMs can generate content that appears credible but is factually incorrect due to their limited ability to understand truth and verify facts. If you question the model about something that doesn't exist or something it does not know - it is particularly likely to answer your question with a hallucination. Hallucinations happen because either:

- the model is unable to find relevant data
- the model does not know which data to refer to

This makes them inadvertently prone to spreading misinformation.
Comment: the definition of hallucination was repeated a few times.
The prompts shoud be: Translate the following from English to Italian. Limitations of LLM(s)

- **Hallucinations**: when the model lacks the needed knowledge or context to respond. How to reduce hallucinations (link): for example ChatGPT tried to summarize and article based on a non existing URL.
  - Cleaning the data. Collect them from diverse and trustworthy sources
  - Tune the inference parameters: temperature, frequency penalty, etc.
  - Use prompt engineering. Example tell the model not to respond if unsure.
  - If it still hallucinates, use RAG, or otherwise Deep memory
- **Access to latest information**: models are trained with specific datasets, that were collected in a specific period, but the models are deployed for extended durations. Called *Knowledge Cut-Off*.
- **Limited Context Window Size** the context window represents the number of input tokens the model can process simultaneously. Increasing the size enables the model to process and comprehend longer sequences simultaneously, offering a deeper understanding of the context. This is particularly beneficial when inputting a substantial amount of specific data into a language model and posing questions related to this data. For example, when analyzing a lengthy document about a particular company or issue, a larger context window allows the language model to review and remember more of this unique information, resulting in more accurate and tailored responses. Historically, there was a quadratic component to the relationship between the model's memory consumption and the sequence length.
- **Fine tune on high quality data**. Constitutional AI. RLAIF: Comment: go into more details about CAI and RLAIF
- **Stay up to date on the latest research**
- **Limited reasoning capability and Industry Expertise** a key reason LLMs perform poorly on reasoning-like tasks is that there is very little reasoning data on the internet. Humans always skip to the key points when writing up their ideas and don't write down their full inner monologue with every thinking step—so the LLM thought it was supposed to just guess at these random leaps from token to token.
- **Benchmarking and Evaluation**
- **Latency and cost**: when the model size and capability increase, so does the latency and cost.

2.4 Choosing your LLM: Metrics, Benchmarks

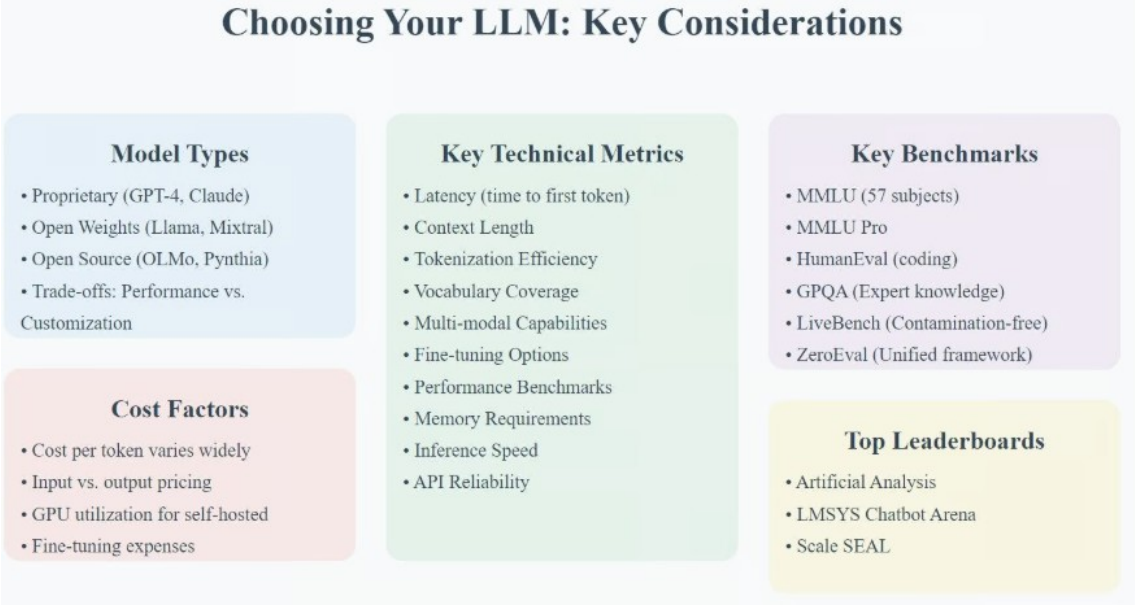
- Proprietary models: accessible only through API or web interface: GPT4, Claude 3 Opus, etc.
- Open Weights: model architecture and weights are available: Llama 3, Mistral, etc.
- Full open source: they provide training data training code,.OLMo, Pynthia. They allow full recreation of the results.
- Some services like Together.AI provide access to open source models.

Open source and open weights models require significant implementation effort and computing resources.
Proprietary models have different cost per token.. Cost per Input token is different than output token cost. *Assume a ratio 3/1 between input and output number of tokens*. Key aspects to evaluate in a model:

- Reliability: if the service is down, your application is down Evaluate the use of a model executed locally
- Privacy and security: consider proprietary models

Live Bench benchmarks
In LLM benchmarks, data contamination is the fact that some data from the benchmark can exist in the model training data.
Benchmarks:

- MMLU multitask accuracy and knowledge, with zero shot and few shot settings
- MMLU Pro
- Artificial Analysis provides leaderboards that rank LLMs based on quality, output speed (tokens per second), latency, price, and context window, offering a comprehensive performance overview.
- Chatbot Arena (LMSYS) Users chat with two anonymous models side-by-side and then vote for the preferred model based on their experience and responses. Model names are revealed after voting.
- Scale Seal: uses private datasets, so they do not become part of the LLM training dataset.



2.5 What is Prompting? Talking with AI Models.

Prompting: how humans talk to AI. Usually using words. AI can understand words, but differently than we do. Prompt engineering: the task of translating from human language to model language. Finding the prompt that yields the best results from the LLM. The Prompt Engineer is the translator from human language to AI language.

2.6 Prompting 101

The user input to an LLM, in broad sense is a prompt. It is a way to tell an AI agent what we want and how we want using human language. Prompt engineering often requires an ability *to give clear and concise instructions as to what you can think of as a relatively novice "assistant" or "intern," where you preemptively think through ways they may make mistakes or get off track*.

2.6.1 Zero shot prompting

The model is asked to produce output without examples demonstrating the task. The LLM can only rely on its training data.

2.6.2 In-Context learning and few shot prompting

The prompt contains demonstrations and examples, then the request. Few-shot prompting contains a few examples; each example contains the desired output for a given input, then a similar question is answered, or similar questions. The LLM(s) struggle with few-shot when the task is hard.

2.6.3 Role Prompting

The prompts instructs the LLM to assume a specific role or identity for task execution. E.g.:"You are a marketing expert. Create a compelling product description [...]".

2.6.4 Chain Prompting

Chain prompting is writing a series of prompts (request & response)

2.6.5 Chain-of-Thought Prompting (CoT)

- Presenting examples of the desired reasoning process (optional, this is few-shot prompting)
- Asking the LLM to explain its logic in its response.

CoT is beneficial for arithmetic, reasoning and thinking tasks, CoT is less meaningful for other kind of tasks. Its performance varies across models.
CoT offers several advantages. Firstly, it simplifies complex tasks by enabling the LLM to break down challenging problems into more manageable steps. This feature is valuable for tasks requiring calculations, logical analysis, or multi-step reasoning. Secondly, CoT can guide the model through a series of related prompts, fostering more coherent and contextually appropriate outputs. This can result in more precise and practical responses, expecially in tasks requiring a thorough understanding of the problem or subject matter.

2.6.6 Tips for Effective Prompt Engineering

Link: Prompt Engineering overview (Anthropic)

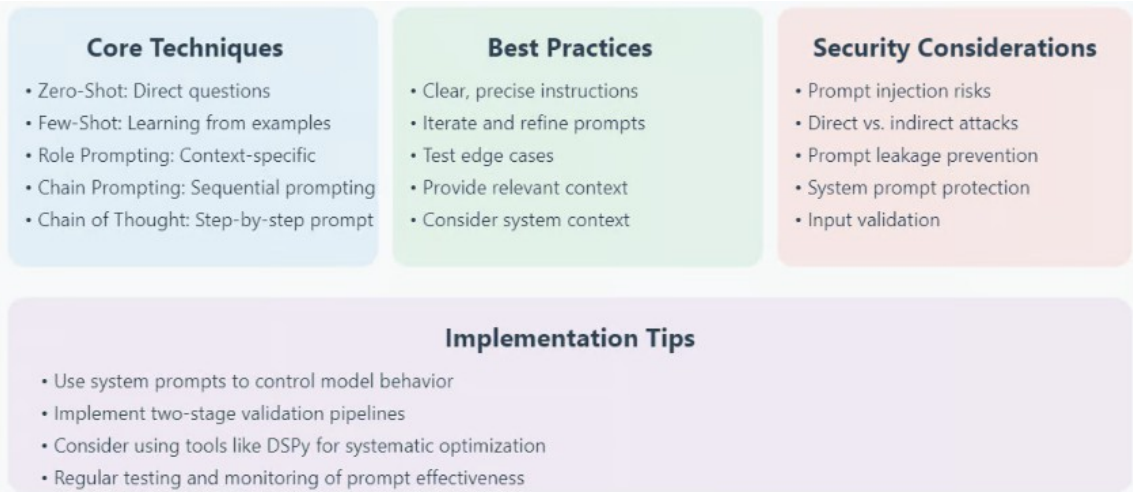
- Communicate clearly and precisely, do not make assumptions, provide all the needed information
- Break down tasks into bulletpoints
- Be clear and concise. Do not introduce unneeded complications

- Be ready to iterate quickly. Multiple refinements in quick succession
- Consider uncommon scenarios: how will your prompt perform?
- Test prompts using imperfect or messy input provided by users
- Analyze the model's responses carefully, study model outputs extensively.
- Treat prompt creation like coding. Use version control to track and manage your experiments.
- *Ask the model itself to point out unclear parts of your instructions*. Similar: *ask the model to repeat back what you asked in its own words*.
- Balance typical cases and edge cases
- Think about the broader system your prompt is a part of. Consider: data sources, latency, and how the prompt integrates with other parts of the system.

Establishing a clear context, providing examples, and using precise wording typically lead to more targeted outputs. Overly general prompts can lead to correct but irrelevant answers, and in some cases, vague prompts may even result in false information. Consider using DSPy: a framework for programming LLM(s) (DSPy docs).

2.6.7 o1 and "Reasoning model" prompting

Reasoning models do not use the iterative exchanges like chat models. Provide comprehensive and detailed context. Include all relevant information and describe how the response should look like. *Role prompting and chain of thoughts are no longer needed with o1*.



2.7 What is Prompt Injection? Can you hack a prompt?

Prompt injection, jailbreaking. There are hacking and defense techniques.

- Prompt injection: overriding developer instructions with malicious input. Concatenates trusted and untructed instructions
- Jailbreaking: bypassing safety mechanisms, to perform actions that are usually restricted

https://github.com/Cranot/chatbot-injections-exploits
https://learnprompting.org/docs/prompt_hacking/introduction

In a real system, set up a "controller" LLM model that controls the output of the model's output. The controller LLM model is instructed to classify the response of the main model as appropriate or not.
It is called two-stage pipeline. The first stage is the main model, the second is the controller model.

2.8 Evaluating and Iterating Prompts

If a prompt works well with a given model, it does not mean that it works well with a different model.

In case a proprietary model is used, its behavior can change over time. So, prepare a set of test cases to evaluate whether the behavior of the model is changed in a way that impacts the developed system.
In case an open source model deployed locally is used, there is no way that the model can change.

Given the dynamic nature of proprietary LLMs, it's important to regularly evaluate their performance with your prompts. This involves generating responses using your prompts and assessing them based on metrics relevant to your application. For example, you might want to ensure that the LLM maintains a consistent text style, like using Markdown correctly if your application requires that. Or, you might want an LLM response in a specific format. One effective way to conduct this evaluation is to use an LLM as a judge, designed to assess the generated responses against your specific criteria and what the response should be like. This can help measure the effectiveness of prompts and ensure that the LLM continues to meet the needs of your application, even as it updates.

Example: checking the output format. A model is used as judge of the output of the first model. The temperature of the judge is set to zero.
Another best practice in evaluating and iterating prompts is maintaining a version history of all the prompts you use. By keeping track of each version, you can easily revert to an earlier version if a new one proves less effective in specific cases.
It is straightforward to use git, but git does not allow you to match a given version with its results: a prompt with its evaluation.

3 Part 1: Building our RAG AI Tutor – RAG Basics

3.1 What is RAG?

https://www.youtube.com/watch?v=LAfrShnpVIk&t=5s
You must have a dataset, which you use to reply to user questions. You must make sure that the answer is composed only using

information from the provided dataset, not from previous knowledge of the model.

- Validate the question
- Find sources in the database = the dataset
- Provide the sources to ChatGPT and create the reply to the user.

- The content is split into chunks of text, e.g. 500 characters
- Each chunk is given to an embedding model, the embeddings for this chunks are created and saved into memory
- Given a new question: embeddings are created from the question
- The question embeddings are compared to those stored in memory
- We look for the most probable content in memory
- The prompt to the LLM is so composed by: the user question and the relevant content retrieved from the memory

You can update the knowledge base as often as you want. The system can also cite the sources it used to anwser the question, so there is an explanation of why it gave that answer.

There are additional points to consider:

- How to determine whether to answer a question or not (the question can be about a topic not contained in the datbase)
- if the question is relevant, or if the answer can be found in your documentation
- Understand new terms and acronyms not in the LLM knowledge base
- Find the relevant information efficiently and accurately

Fine tuning is an alternative to RAG. RAG is relevant, with or without fine tuning.

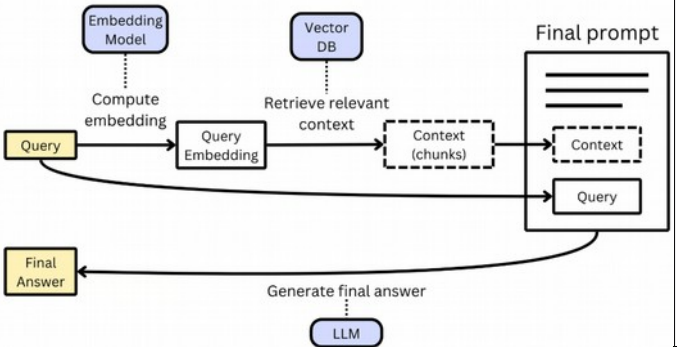
3.2 RAG 101

Challenge: optimize the selection and utilization of each text chunk to enhance the performance of the LLM. The core chalenge is carefully filtering and curating the most relevant information to fit within the context window of the LLM.

3.2.1 How does RAG work?

- Retrieve relevant information from a database or corpus
- use the information and the user question to compose the prompt.

RAG leverages the LLM few-shot learning capability. Paper about RAG: https://arxiv.org/abs/2005.11401v4



YouTube video

An embedding model is used to encode the source of knowledge. The same embedding model encodes the user's query. It must be the exact same.

Note that an embedding model can also be used to detect anomalies. It means that the question is unrelated to what is stored in the vector database.

The embedded user query is used to fetch the related documents. The most closely related documents must be selected, and the most closely related sections in the document. The RAG must adapt to a limited context window.

Strategies related to RAG:

- Chunking: dividing documents into smaller segments
- Annotating documents with keywords to streamline retrieval
- Effective prompt augmentation techniques

Given that the RAG adapts a LLM to a specific user need, try to incorporate industry expertise into the RAG design.

Technology stack:

- Embeddings: OpenAI `text-embedding-3-small` model
- Chroma DB database
- LlamaIndex

Options:

- Create your framework from scratch if you have time, resources and you are developing a long-term product.
- OpenAI Assistants: GPTS
- LangChain: integrates with vendors. Supports Prompt Engineering, and Structured output parsers. It manages user context throughout the application, in sessions. Balance between customization end ease of use.
- LlamaIndex: Data handling; data loaders from different sources. Efficient indexing and retrieving without calculating embeddings for all. Complex datasets.
  - Efficient index and retrieval methods
  - better chunking methods
  - Multimodality
  - Recursive retrieval system

3.2.2 Cache augmented Generation (CAG)

Preloading knowledge directly into the LLM memory. Uses Key-Value cache.

- CAG is fast: no need to search through documents anymore.
- Reliable, but there is the risk of adding irrelevant information to every query.
- It is simple.

Downside of CAG:

- you are limited by the size of the context window.
- Super efficient, but costlier, because you always sent a lot of context every time, even for simpler queries.
- Not finding the relevant bits.

3.2.3 Building a Basic RAG pipeline from scratch

Given a corpus, each document is divided into chunks, e.g. 1024 characters.

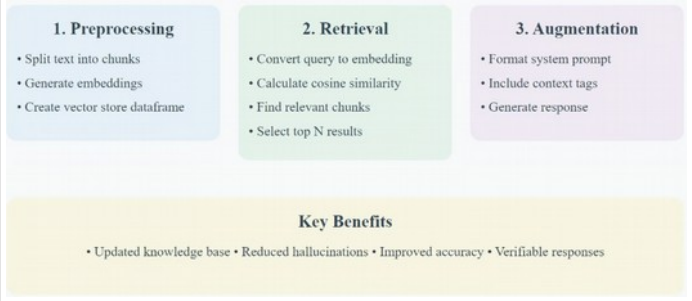
- For each chunk, the embeddings are calculated
- each chunk is stored along with its embeddings

Given a user question:

- embeddings are created from the user question
- the user question embeddings are compared with the corpus embeddings. They are ordered by cosine similarity. The last items are retrieved. The number of chunks to retrieve is 3. The number of chunks to retrieve depends on the application. Too few and the LLM can miss additional chunks that can provide additional information. Too many can confuse the LLM and increases the number of input tokens (the cost).
- The retrieved chunks and the user question are composed in a prompt that is fed into the LLM.

3.2.3.1 Choosing the best temperature parameter

Decreasing the temperature makes the response more deterministic e.g. zero.
Increasing the temperature introduces creativity and randomness in the response.
Temperature is a tuning factor that controls the diversity of potential outputs. Increasing the temperature, the models selects fom a broader set of possibilities, including responses that are more nuanced, unconventional or interpretative.
With low temperature, the model might regurgitate pieces of the prompt (or the training data) verbatim.
So, low temperature may produce repetitive responses, or lacking variety.
Low temperature for tasks that require concise, factual responses, while a moderate or high temperature may work with scenarios where exploratory anwsers ar desired. Temperature sets the balance between reliability and variability.



Building a RAG pipeline, from text to knowledge.

4 Part 1: Building our RAG AI Tutor – RAG with LlamaIndex

4.1 Basic RAG with LlamaIndex

The two most commonly used frameworks to help build RAG projects and LLM pipelines are LangChain and LlamaIndex.
Documents used in RAG are divided into chunks: from each chunk the embedding is created.
Chunks have a size, in tokens, a useful technique is having overlapped chunks. Some text is found at the end of a chunk and at the beginning of the next one. Documents are divided into chunks, so that the LLM request does not include all the document, but rather just the relevant chunk. This is because LLM context size is limited. Even if it is increased, adding more content means extra time and cost to process the prompt.
However, this approach introduces a trade-off between the size of the chunks and their context. Smaller chunks result in more precise embeddings because they allow the model to focus on a specific piece of information without the noise of unrelated content. On the other hand, larger chunks encapsulate more of the document's overall context, potentially offering a richer understanding of the text at the cost of precision. To mitigate the downsides of both approaches, chunk overlap is employed.
Example: chunk size of 728 tokens and an overlap of 64.
Choosing the optimal chunk size for a project involves a bit of trial and error, with a general guideline suggesting a range between 300 and 1000 tokens. Chunk size between 300 and 1000 tokens.

4.2 How Vector DB(s) work and when to use one

A traditional DB is like a spreadsheet, or a table, there are rows and columns. To retrieve somethink, there must be perfect match, example to the row key.
Vector DB gives results based on similarity: vector embeddings are numerical representations.
Each item stored in the DB is associated to a vector index. Data is retrieveed based on how similar it is to the question we are asking. If the user asks a question the question is converted into a vector, then the Vector DB retrieves the most similar items, based on vector similarity.
Vector index: the vector after the indexing process has been run. Suppose the Vector DB only contains images of cats. If I ask for a tiger it will retrieve a cat. People have added other techniquest to vector search to avoid retrieving unrelevant data, using the old-school approach, called keyword filtering.
Keyword filtering (pre/post)can be done before vector search: this is called Pre-filtering. Problem: I may filter out relevant data that the vector search is then unable to find. Extensive filtering slows down the process.
Post-filtering: keyword filtering after vector search. After vector search, keyword search must still filter out irrelevant data.
A vector index is a search mechanism, used to filter a database. Vector DB(s) do not just do search. They also provide database management capabilities: better access control, every user can access to differend data. In order to do that, the DB must access the metadata to filter data appropriate to each user. The systems ust keep running even when something goes wrong.
Fault tolerance must be provided. The Vector DB runs in different nodes, so that if one fails, the other can take over the service. **Sharding**: data is split into smaller pieces called shards. When you run a query, the query is handled by a shard router, The results from the different shards are then added together to create the result for the user.
Reliability and ease of use, e.g. API.
How to choose the right Vector DB? Characteristics to look for:

- Performance
  - How many questions / queries processed per second?
  - Latency: how quickly are result returned?
  - Speed of data insertion

- Indexing method: understand the trade offs and whether they fit our project needs.
- User friendlyness
  - API support?
- Scalability: as data grows, the Vector DB must scale with it.
  - Vertical scalability: adding more powerful hardware
  - Horizontally: adding more nodes to the system
- Pricing

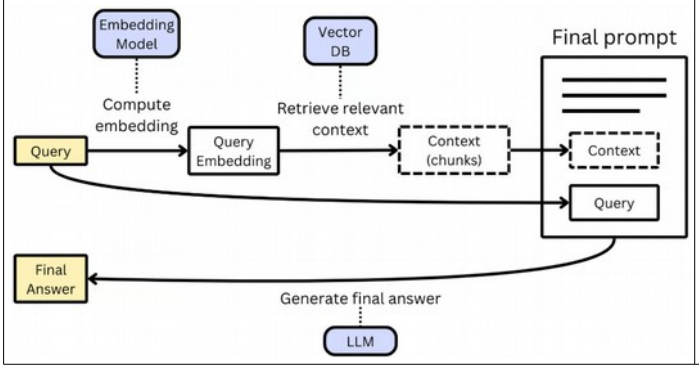
Best practices in implementing Vector DB(s)

- Use built-in preprocessing tools to preprocess data, if available. You may need to filter and clean data, in some cases data must be normalized: scaled to a common range
- Optimize queries: batch queries. For example, apply metadata filtering before vector search.
- Distribute queries across processors (i.e. nodes) for speed
- Cache frequent queries
- Monitor the Vector DB performance. Regularly monitor key metrics for database health. Metric examples latency and throughput. Latency: the time between a request and a response. Throughput: how many queries are served per second.

Serverless vector databases: DB(s) have high computational and storage cost. Serverless databases: they separate computation and cost by using geometric partitioning. This breaks the index into smaller pieces, so that searches are more focused and computational resources are used only when needed, cutting down costs. Sometimes this increases the time to get the response. Another challenge is handling multiple users on the same HW, without wasting resources.
If you add a new datum, it must be added to the index, it is not immediately available to all users. Some systems use a Freshness Layer, a temporary cache where new data is stored, in order to be immadetialy available to all clients/users.

4.3 Using a Vector Database

Traditional databases and methods of storing data fall short when handling and querying the high dimensional data points used in RAG pipelines; vector DB(s) have emerged as an efficient solution. Vector DB(s) are specialized data structures tailored to efficiently manage, store and retrieve high-dimensional vectors.
Embeddings: vector representations of various form of data into numerical format. Embeddings are powerful because they represent semantic similarities in a measurable way.
Some methods are based on embeddings: Approximate Neares Neighbours (ANN), and Hierarchical Navigable Small World (HNSW). Vector DB(s) use smaller, custom models to generate embeddings. They do not use the same embedding models as the LLM(s)

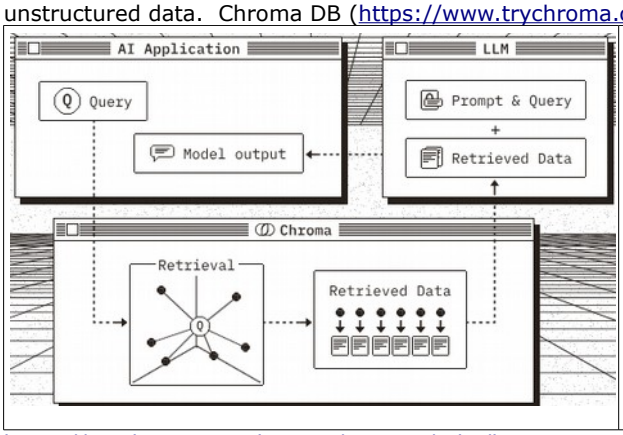


Role of the Vector DB in RAG: the Vector DB stores the data sources as vector embeddingsand retrieves the relevant context based on similarity search. The retrieved relevant contexts are determined by:

- the vector search
- the vector indexing.

The Vector DB handles both of them.

Vector DB are for use cases involving high dimensional vectors, like semantic search. They are scalable, making them suitable for large scale applications, they can store multimodal data: text, images, audio and video. They can handle structured and unstructured data. Chroma DB (https://www.trychroma.com/) is used in the lesson.



Chroma DB features: multimodality, metadata can be added to vector embeddings. Bindings to Python and Javascript/Typescript. With Python, Chroma DB can be used in memory or in client/server mode.
Other options are:
Deep Lake: (https://docs.deeplake.ai/latest/)
Weaviate(https://weaviate.io/developers/weaviate) Pinecone (not open source). Key criteria to make the decision:
- Search type (keyword or semantic)
- Scalability
- Retrieval latency
- Real-time index updates
- Ease of use
- Available ecosystem support
- Pricing structure / pricing model
- Additional data storage needs: some vector databases have a broader platform that can also store data useful for model training or other corporate use cases

https://academy.towardsai.net/courses/take/beginner-to-advanced-llm-dev/multimedia/59791115-using-a-vector-database

Llama Index calls index the data structure that allow to access the underlying dataset, e.g. the embeddings. In Llama Index there are multiple index methods available, but Vector Store Index is the recommended. Other are Summary Index and Tree Index.

4.3.1 RAG Evaluation Techniques

Evaluation metrics and techniques for RAG systems. RAG combines retrieval mechanisms and generative models, like GPT-4. Testing yourself, based on your impression is called "vibe checking".

RAG:

- Retrieval
- Answer generation techniques

4.3.1.1 Retrieval metrics

Retrieval metrics: how well the system retrieves the relevant information from the dataset. There are four metrics:

- **Precision**: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$ measures of the information retrieved is relevant
- **Recall**: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$

  In the denominator, False Negatives are the documents that it should have retrieved, but it did not. Recall: the proportion of retrieved data over the set of data it should have retrieves. Recall: if it recalls all.
- **Hit Rate**: the proportion of queries for which at least one relevant document is retrieved within the top few results. Whether the system is able to find relevant data at all.
- **Mean Reciprocal Rank (MRR)**: it is one of the most useful metrics. Useful where users are interested in relevant results in the first result, like in a web engine.

  $$\frac{1}{Q}\sum\frac{1}{\text{Rank of the first relevant document}}$$
- Normalized Discounted Cumulative Gain (NDCG): it gives a more holistic evaluation of the ranking quality, considering relevance and order. Ideal for recommendation systems where the entire list is important, e.g. top 10 list.

4.3.1.2 Generation Metrics

- **Faithfullness**: the integrity of the answer: whether the response contains true facts, considering the retrieved context. Example: the context says that Einstein was born on 14th march 1879, but the response says it was born on 20th March 1879.
- **Answer Relevancy**: whether the response contains information relevant to the question.
- **Answer Correctness**: assesses whether the answer aligns with a given query reference answer. Useful when, for every query, you have ground truth for comparison.

4.3.1.3 Evaluation Dataset

There must be an evaluation dataset to run them on. The evaluation dataset is produced by domain experts.
Suppose we have all the RAG system in place. How to judge whether the answer is good.
One option is using an LLM model as a judge. This approach is better when we produce two responses and let the LLM judge which one is better. This is called **pairwise comparison**. When you use this technique, you must randomize the order of the answers you give to the LLM, because LLM(s) seem to have a bias with order. Moreover give the LLM the option to declare that both anwsers are equally good (tie).
The best thing is using a Chain of Thought approach when evaluating. The model must explain why is it preferring a given answer. To the thought process is added to the context that the LLM uses to produce the result. Make sure that the answers have about the same length. Because length influences the perceived quality.

Moreover, you must understand your users' actual needs. A way to do that is logging and analyzind user queries. Analyzing user questions.
You must then implement the evaluation process. For example use Ragas (https://www.ragas.io/) to implement the evaluation process. Third party frameworks already provide the described metrics.
Take also into accout human evaluation: fluency, naturalness of responses and usefulness in real world scenarios are important.

4.4 Evaluating your RAG pipeline

In the jupyter notebook, it makes sure that every node has always the same id. By default, Llama index generates a random node Id. Evaluation focuses on the retriever and the generator.

- Correctness: whether the answer is aligned to a reference answer
- Faithfulness: if the answer correctly extracts data from the context.
- Context relevancy: is the answer contains relevant data requested by the prompt.
- Guideline adherence: criteria can be style, factual, ethnical, etc.
- Embedding Semantic Similarity: this technique involves calculating a simiality score between the generated and reference answers' embeddings. It requires reference labels to estimate the closeness of the generated response to a reference in terms of semantic content.

4.4.1 The Golden Dataset

https://github.com/microsoft/promptflow-resource-hub/blob/main/sample_gallery/golden_dataset/copilot-golden-dataset-creation-guidance.md
The golden dataset is a collection of pairs:

- realistic customer question
- expert answer

At least 150 pairs, not less than 100.
For each question, an LLM response can be generated, to feed the expert.

STEP 1		STEP 2 (optional)
(mandatory) In this column, provide a set of questions similar in nature to what you expect or know customers would typically ask. Ensure each question: - Is self-contained (subdomain should be clear from question) - assumes no further context These questions will get sent to the AI models as-is.	(optional) Alias of person that crafted the question	(optional) To craft your expert answer to the question in column A, you might want to use an LLM. This is just to help you get started on an answer. This column can be used to record the LLM answer, *before* human review. By no means should an LLM answer be used without proper supervision and careful review/enhancement/correction by a human expert.

STEP 3			STEP 4 (optional)
(mandatory) In this column, record the expert answer to the question in column A. This answer should be correct and complete, crafted by an expert. (Optionally the starting point can be an LLM answer as recorded in STEP 2/column B.)	(optional) In this column, record the links that should contain the answer to the question. This can be more than one link; split those with ALT-ENTER. This column informs the model quality scoring on where the answer ideally should have come from (grounding).	(optional) Alias of person that crafted the expert answer	(optional) The expert answer is double-checked by a reviewer to have a second opinion. Record the alias of the reviewer and the date of last review.

Make sure that each question contains enough context to give an answer.

5 Part 1: Building our RAG AI Tutor – Using LLM(s) and Embedding Models

5.1 Privacy and Compliance concerns for LLM(s)

Enterprises skeptical of LLM API endpoints from big-tech companies are often already storing their internal documents on these same cloud platforms.

- LLM(s) can memorize and reproduce data they were trained on, or exposed during operations.
- Lack of fine grained access control: many LLM(s) API do not provide robust access control. It is difficult to control who can access or manipulate specific information.
- Compliance to data privacy regulations or internal company policies, for example the GDPR. There is the right to delete personal information, but such knowledge cannot be deleted from a trained model. California AI regulation Bill (SB 1047) Likely, further regulation will target AI applications. Such regulations may slow down development process, and failure to comply may result in fines and pther penalties.

Key Security concerns in LLM(s)

- Prompt injection attacks: malicious input can bypass safety measures and produce harmful content, or extract private information. Prompt injection is overriding the original developer instructions.
- Jailbraking: bypassing the safety mechanisms of LLM(s) to make it perform actions it was not intended to do.
- Prompt and Data Thieft: using prompt hacking and jailbreaking to steal LLM system prompts from your product, or steal the data you use for RAG.
- API key exposure
- Legal and Contractual Risk from Errors and Hallucinations. It is currently easier to build a product or tool used internally in a company. External consumer-facing products require some extra guardrails as it is hard to train your end users on LLM weaknesses—but here, too, we see many high-value use cases already.

Security and Privacy

- Public facing Chatbots: e.g. ChatGPT from the web interface. They are considered the least secure. Prompts can be used for training.
- Public LLM API Endpoints: requests can be batched with other users. Sharing batches with other users means that GPU(s) can reach maximum utilization. The service quality is subject to fluctuations. Some vendors let you apply for *"zero logging" Prioritize providers with robust security features, use "no-logging", limit prompt retention*. Regularly review the provider's terms of service and privacy policies for updates.
- Dedicated API Instances: some LLM API providers offer LLM(s) on dedicated API instances at a higher cost. Those instances are hosted on Azure, or Google Vertex, etc. More reliable service and stricter data segregation, still the complexity of installing is on the vendor. Superior data isolation and control
- In House Deployment on Major GPU Cloud Providers: you must take care of installing on the Cloud.
- Local In-House LLM Development

Check the code for backdoors. Keep track of dependencies and minimize them. It is possible to run the model on a dedicated VLAN without external internet.

Mitigating Privacy Risks in LLM Deployments:

- Minimization: minimize the data you provide to the LLM, to reduce costs and risk of hallucination. Only include the information strictly needed for the task
- Anonymization: protect sensitive information, for example before sending data to third party LLM(s): strip personal details and addresses. Take into account that this process can also reduce performance.
- De-identification,

Continuous monitorinf and auditing. Regularly review logs, access controls and scan for misconfigurations.
Minit external access. Setup an Incident response planning.

Open source vs Third- party API(s): a privacy and security comparison

Points to consider:

- Data sensitivity: does your organization handle sensitive data (healthcare, finance, government)?
- Compliance requirements: are you subject to privacy regulations like GDPR or HIPAA. Is there a data deletion policy?
- Cost versus privacy
- Technical expertise

5.2 Using Llama 3.1 70B on Together AI

5.3 Multimodal LLM(s) and Their Role in RAG Pipelines

Multimodal LLM(s) = MLLM process diverse data types
Usually they convert different types of input into an unified format, using specialized encoders. All comes down to tokens. There is an advanced attention mechanism to analyze relationships between different modalities.
Applications in RAG pipelines: for example documents with tables and image: traditional text models miss the information contained in images. MLLM(s) can be used in a preprocessing step where PDF(s) are converted into text and images are

described. Images can be embedded alongside text.
There are also voice-enabled RAG systems: the operator can talk to the system, the audio is converted into text.
Challenges and considerations:

- MLLM required greater computational resources
- data preparation: consider if preprocessing steps must be performed to feed multimodal data
- More expensive, less mature models and ecosystem, compared to text-onyl LLM(s)
- additional ethical considerations
- pipeline changes if there are existing traditional RAG(s)

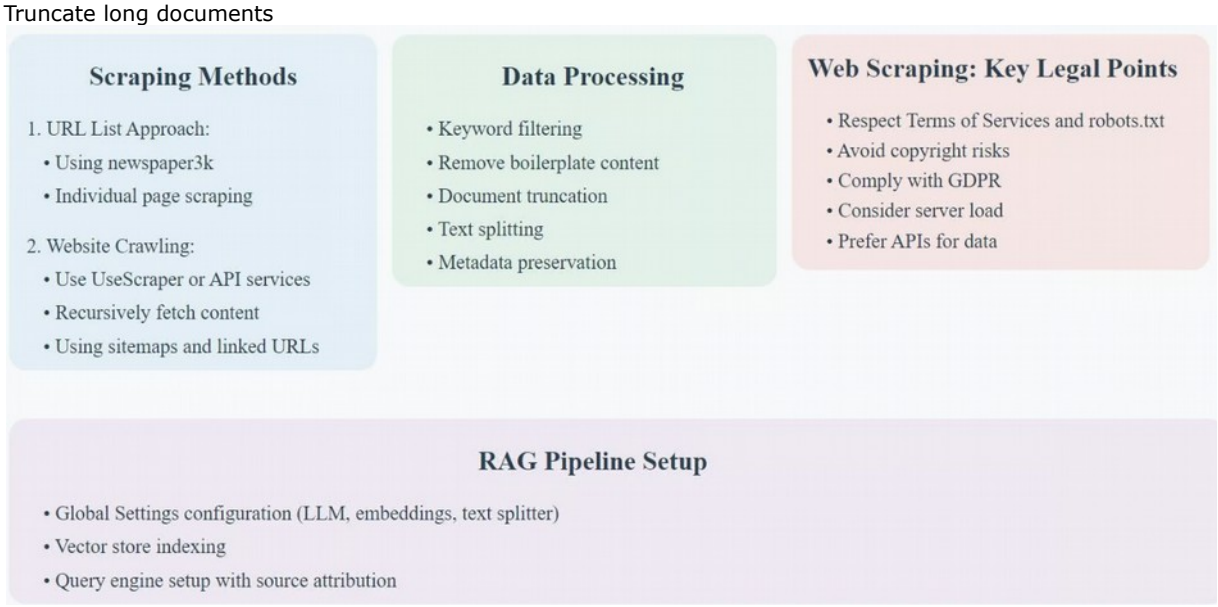
Typically, today's MLLM convert images to text

6 Part 1: Building our RAG AI Tutor – Collecting, Filtering, Cleaning Data

6.1 Scraping Data from Websites

The course uses Beautiful Soup to parse HTML, newspaper to download the HTML from the site (does not work with dynamic sites)
There are services like UseScraper that can crawl a website,
Data cleaning techniques:

- keyword filtering: remove the parts that do not contains a set of desired keywords
- remove boilerplate content like headers, footers, navigation menus. Beautiful Soup can be used to clean the HTML.
- Truncate long documents



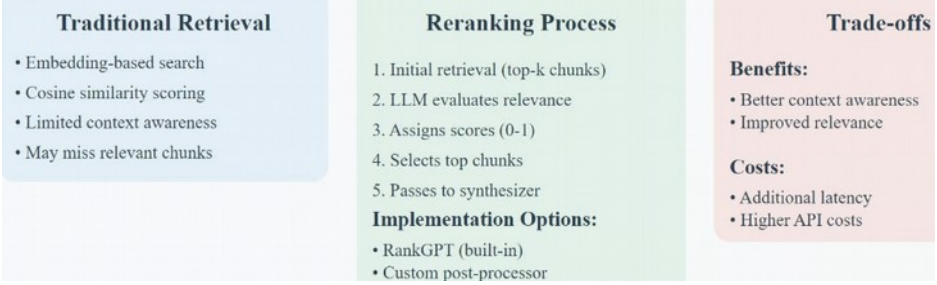
A lesson demonstrates how to use Firecrawl to scrape a Web site.
Perplexity Web API: a RAG-based LLM pipeline that powers a general web search engine.
Llama Parse can be used to parse data (it is a service, it does not run locally).

7 Part 1: Building our RAG AI: Advanced RAG

7.1 Larger Context Window: More Chunks

7.2 LLM as a Judge: Ranking Chunks

RAG uses embeddings, but embeddings do not have the rich understanding of natural language as the LLM. Cosine similarity is more intelligent than keyword search, but less intelligent than LLM(s). Paper: https://arxiv.org/abs/2306.05685 .
A primary criteria is whether the answers are relevant to the query. Note that LLM inherit the biases of the dataset they were trained on.. Different approaches: one approach uses RankGPT
Reranking techniquest are used when the retrieval phase (from the vector database) retrieves many chunks of data, that cannot be feed all to the LLM. These techniques are more expensive and introduce additional latency.



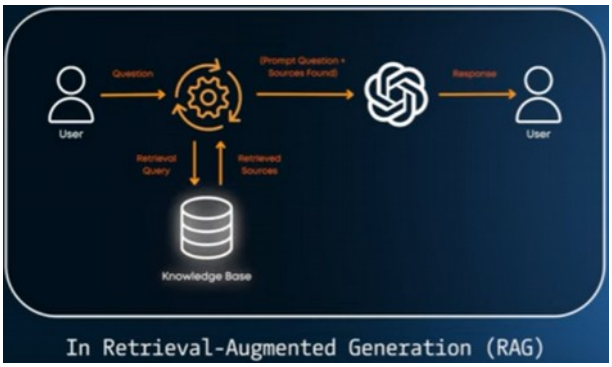
7.3 Boost Your RAG Searches

Keyword search: it find exact of close matches to the search terms. Best Matching 25 (BM25) and BM42 are populae algorithms

for keyword search. BM25 gives high weight to rare words in a document TF Term Frequency and (IDF) Inverse Document Frequency. It uses a saturation function to make sure that if a term appears too often it does not appear automatically relevant. Keyword search works when there is a need for quick and exact results. Great for well organized data and specific terms to search. Those algorithms do not take into account of synonyms.
Embedding search / Semantic Search: Allows to search based on conceptual similarity. It needs more computational resources and sophisticated models.
Hybrid search combines keyword and embedding search:
GraphRAG: it understands how data is connected. It is about understanding how data is connected. It uses knowledge graphs. It extracts entities and relations, capturing semantic connections between data. We use GraphRAG when the data has complex connections. For example in the legal sector. It is powerful, but slow.
Metadata filtering: filtering based on information and metadata. When data has clear labels or users want to have more control over their search. Metadata filering occur before vector search. Metadata filtering restricts the search space of the vector search.
Router uses the best search method

- simple, organized data => keyword search or SQL
- meaning or context => vector
- a bit of keyowrd seatch and vector ? => hybrid search
- complex, connected information? => GraphRAG

7.4 What is indexing? Indexing methods in Vector retrieval



How to prevent the retrieval part of the RAG to slow you down?
In RAG, the user query is transformed into an embedding and compared. What does happen in the Vector DB has millions of entries?
The Vector Index is a sorting algorithm: is the algorithm that compares the embedding from the user query with the embeddings stored in the DB, it must be fast and accurate. Distance metrics ,e.g. cosine similarities. Different methods to create the vector index:

- Flat index: highly accurate but slow
- Locality Sentity Hashing: Vectors are griuped into buckets based in similarity. Similar index are grouped using a hashing function.
- Inverted File Index: similar to LSH, but we use clustering to create buckets. When a query land on the boundary of a cluster, we then search in all of them.
- Inverted file with product quantization (IVFPQ) speeds up the comparison.
- Hierarchical Navigable Small Worlds (HNSW): it is fast and accurate. It must create a hierarchy of buckets. (*Qdrant uses HNSW*).

To improve: you can use prompting techniques to add context to the user question. Moreover, it is possible to implement keyword search or hybrid search.

7.4.1 Hierarchical Navigable Small Worlds (HNSW)

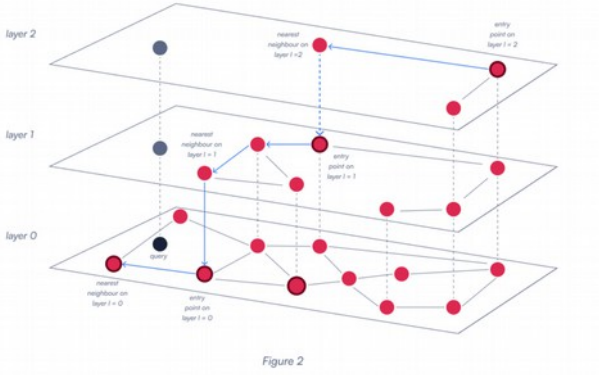


Figure 2

8 Part 1: Fine Tuning

8.1 Optimizing the model: inference and fine tuning

If the LLM is a student that has read all the library, fine tuning is extra tutoring in a specific subject. Trained a bit more on a specific dataset. Fine tuning (FT) is not a competitor to RAG, it can complement it. Downside: FT is complex and expensive. Build the right dataset, understand the fine tuning process, iterate and find the right combination of hyperparameters. There are some techniques:

- quantization: the model runs faster and uses less memory. Precision: how many bits are used to represent a number. Float32 are accurate but heavy in computation costs, int8 and int4 are lighter and faster but less precise. Model do not change very much, while activations are the dynamic output generated by the model, when the model processes the information through its layers. So, activations must be precise. In a model there are more activations than weights. 3:1. Reducing the precision of weights has less impact because weight are relatively stable during the operation of the network. Quantization makes the model run faster, but it can also hurt the precision of the model. There are multiple tricks to make sure that the quantization does not have impact on the precision. Bits and bytes is a popular library. It uses 8 bit and 4 bit quantization.
- PEFT: Parameter Efficient Fine Tuning:

- Adapters
- LORA LowRank adaptation
- Qlora: Quantized LORA.

Post-Fine tuning Optimization:

- Smooth Quant: quantizes both weights and activations to 8 bit. Built into Faster Transform, Amazon Sage Maker, Nvidia Tensor RT and ONNX.
- Gradient Post-Training Quantization (GPTQ): after the model is fully trained
  - it can be applied both to weights and activations.
  - Different levels of quantization are supported.

There are other techniques in fine tuning: example:

- Model Pruning: trimming a tree, makes the model smaller and faster. TF Model Optimization Toolkit and Torch Prune can be used. See Minitron paper by Nvidia. LLM Pruning and Distillation in Practice: the Minitron approach.
- Mixed precision training: fp16, and fp32

Knowledge distillation: training a smaller model
Graph optimization techniques: layer fusion: cuts down unnecessary steps. Similar operations are merged into one.

8.2 Fine Tuning 101

Fine-tuning refers to the process of taking a pre-trained model and adapting it to a specific task by training it further on a specific dataset. This additional training helps the model perform better on the particular task by learning the nuances and specific patterns in the task-related data. Unlike Retrieval-Augmented Generation (RAG) and prompting, this involves changing the LLM's model weights.
Fine tuning makes the model excel in a domain specific scenario. Standard full-model fine tuning requires modifying all parameters in the pre-trained model. Using memory and cost -effective fine-tuning techniques is essential (e.g. LORA).
Fine tuning can be used to create a smaller model, that has a lower latency than the big, original model.
FT is not an efficient first step when improving a LLM product. Try first to improve data sets, prompts, RAG pipelines.

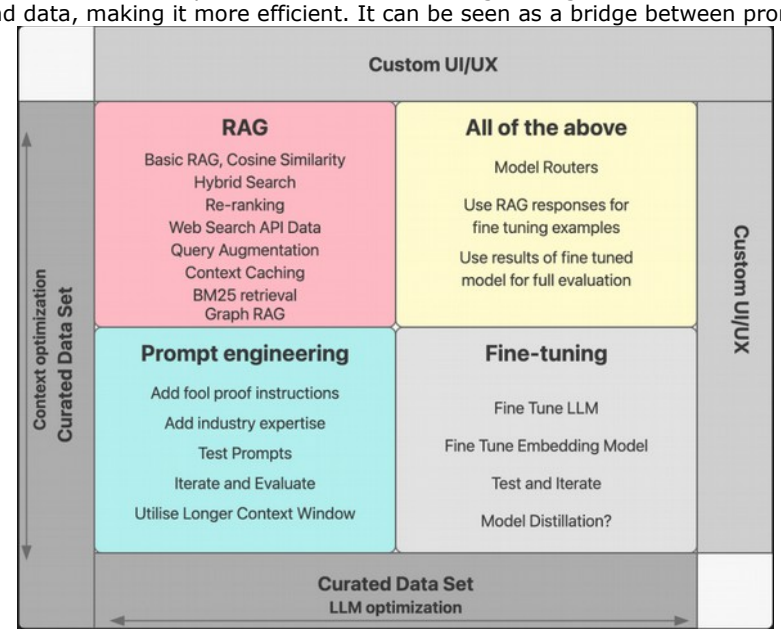
It is possible to build a router model that choose which specific model where to forward each input question.
Google Gemini can be used with fine tuning (https://ai.google.dev/gemini-api/docs/model-tuning)
Techniques for fime tuning LLM(s)

- Instruction fine tuning
- Supervised fine tuning (the dataset is composed of request and response)
- Reinforcement Learning from Human Feedback, similar methodologies are Direct Preference Optimization (DPO) and Reinforcement Learning from AI Feedback (RLAIF)

Standard Full model fine tuning: all the model parameters are modified during the training using task-specific data. Used in case the target task is different from the origianl task that the model was trained on.
LORA or Adapter Layers can be used.

Open source resources for LoRA : PEFT Library (https://github.com/huggingface/peft) and Lit-GPT (https://github.com/Lightning-AI/lit-gpt)
Qlora (example: a 48GB GPU) (https://arxiv.org/abs/2305.14314) tries to conserve memory without sacrifiying performance

Prompt tuning focuses on a small set of task-specific tokens or embeddings that guide the model's response. This method requires less computational power and data, making it more efficient. It can be seen as a bridge between prompting and fine-tuning.



Fine tuning is prone to bias amplification.

8.3 Is Fine Tuning an embedding model worth?

What is an embedding model? Converts text into a vector called embedding. It is a fixed length representation of text, audio, image, etc. When text is converted into embedding, similar meanings are close together. High dimensional representation, also

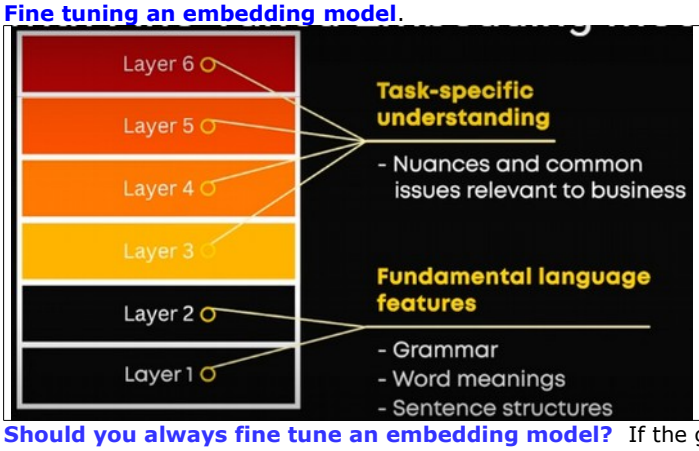
called latent space. Embeddings lie in the latent space. Embedding models:

- undertand relationships between words
- used in text and image search engines, recommendation systems, chatbots, fraud detection systems, etc.

Vector databases are optimized at storing and retrieving embeddings.
Embeddings can be expensive, because each embedding has thousanrds of numbers. High dimensional.
Matrioshka Embedding. So a technique is called Matryoshka embedding.

- You start with a large-size embedding , e.g. 1024
- During training you reduce to smaller and smaller: 512,, 256, 128, down to 64.

You truncate them , e.g. to 64. You search in the 64 space. You compare the 64 dimension, to find the most similar, and you get the ones that are closer.
Then you repeat the search, comparing the 128 dimension with the same vectors. You rerun the search using 128 dim, the again with 256 dime, up to the 1024 dimension.
Fine tuning an embedding model.



Upper layers of the model are more task-specific, lower layers capture the more general features. Steps to fine tune an embedding model:
. select a pre-trained model: a model that already performs well on a general task
. Prepare your data, specific to your task-related
. Adjust the model
. Evaluate and Optimize
. Integrate it with the RAG system

Should you always fine tune an embedding model? If the general embedding model already performs well, there is no need. So, before fine tuning an embedding, evaluate the performance of your current embedding model. If your task requires understanding the deeper nuances of a specific context, and you have valuable information in your database, then fine tuning the model may make sense. Useful if the task requires expert knowledge.

8.4 Fine tuning Embedding Models with Llama Index.

Llama index allows fine tuning various components, like embedding models, adapters, routers. For example fine tuning an embedding model using an adapter layer. An added layer. The dataset is a collection of question-and-amswers pairs.

9 Part 1: Expanded RAG Toolkit

9.1 Non technical skills for Effective LLM Development

9.1.1 Business, Product and Entrepreneurial Skills

9.1.1.1 Considering business strategy and economics.

Total cost of ownership of the solution:

- Training Costs
- Inference Costs
- Data Collection and Curation Costs
- Monitoring and Maintenance Costs
- Human Oversight Costs: in case humans must review model outputs

ROI calculation:

- time saved by end-users
- error reduction rates
- customer satisfactionimprovements
- competitive advantage metrics

9.1.1.2 Communication and Management Skills

9.1.2 New Learning and Development Design Philosophies

9.1.2.1 Agile Learning and Keeping Up To Date

9.1.2.2 System Design Philosophy and Quality Assurance

- Design systems that fail safely when they encounter edge cases. Include confidence thresholds and cases where the LLM is unsure, instead of providing wrong results.
- Allow the user to provide feedback
  - User feedback collection
  - Monitoring Systems
  - Monitoring for emerging use cases
  - A/B Testing Frameworks: use A/B testing to compare different versions of the model or prompts.
  - Develop Evaluation Metrics Beyond Traditional Accuracy Measures: consider metrics like consistency, relevance and adherence to brand voice
  - Managing Test Data Creation and Curation

9.2 Adding Question Validation and Routing

9.2.1 Question routing

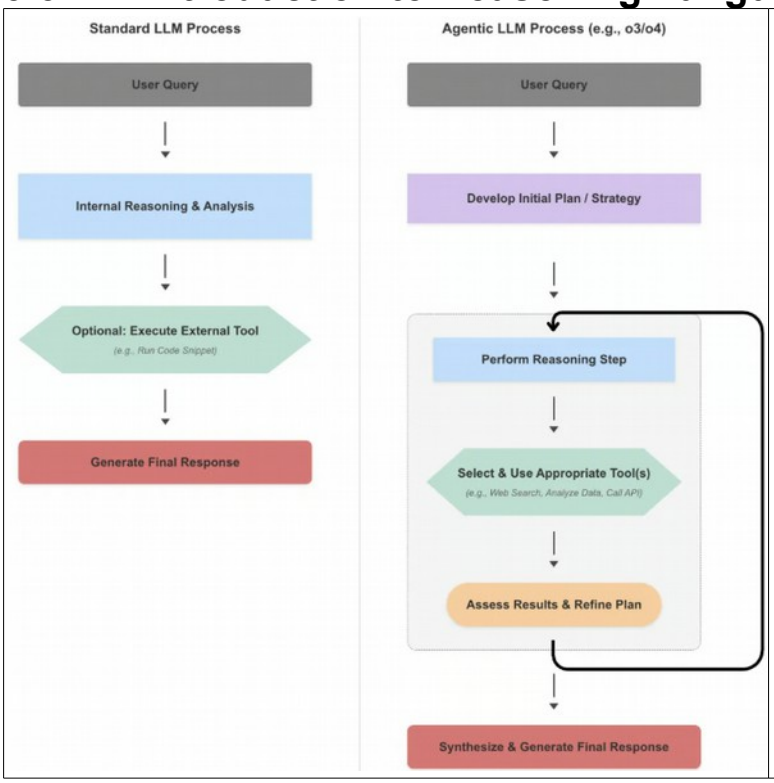
Question routing: used when you have multiple LLM(s) in the system, so that each question is handled by the most appriate model. It is useful when there are distinct knodledge bases. In Mixture of experts: a subset of the model is activated, depending on the query. With Question routing, different models, or parts of the codebase are activated. Or, the question can be sent to different LLM(s) the more or less expensive depending on whether there is a need for more performance.
The question can be sent to:

- the more appropriate vector database filtering the data using metadata or other mechanisms
- multiple tools can be set up, letting the agent to decide who to forware the request to. This way, each LLM is selected depending on its specific strengths. For example a model may excel at coding, another can excel at mathematical computations, etc.

9.2.2 Question validation

Question validation is when the model rejects unrelated questions.
The lesson also uses a Routing Framework called Route LLM: https://github.com/lm-sys/RouteLLM

9.3 An Introduction to Reasoning Language Models



LLMs initially were trained purely at predicting the next token in a sequence, effectively mimicking patterns from vast amounts of text data. To become more useful, integration of instruction tuning and RLHF. At this stage models still struggled with multi-step reasoning. Now the training is aimed not just at token prediction, but to find "full correct solutions to verifiable problems." The model is rewarded when it finds the solution, not the next token. Reasoning models have higher costs and latencies, compared to completion models. Resoning models form a stronger foundation for building autonomois systems (agents).
Resoning models are trained on more data where the solution is verifiable, like math and coding. So these models overfocus on these domains.
Shift to agentic capabilities: example o3

9.4 Enabling Conversational Memory in LLM(s)

Conversational Memory means maintaining the context across multiple interactions. Otherwise each query to a RAG is treated independently. The system must remember the previous interaction with the user, who may need additional clarifications. The system can thus handle multi step tasks or complex queries. The system itself may ask for further clarifications if a query is ambiguous. There are different approaches:

- Basic memory chat: the system keeps track of all messages exchanged, they are added to the context.
- Streaming chat: the user can start reading the response while the system produces it.
- Condense question chat: the conversation is condensed and then used to query the RAG.

- ReAct chat: Reasoning and Action In each interaction, the ReAct agent assesses which of the tools is better to answer the query. This mode is more reliant on the quelity of the LLM.

Suppose that the user asks a question: the system provides a response based on the RAG. In case the user asks a further question, the system may have enough information to answer straight away, without accessing the RAG again, which adds latency.

9.5 Adding Speech With GPT-4o Audio.

About text to speech: there are Text to speech libraries that run locally, e.g.

- https://github.com/rhasspy/piper
- https://pypi.org/project/pyttsx3/

GPT-4o audio model: the LLM can directly reply to the audio. Usually you had to split into different steps:

- Transcribe the audio into text
- Query the LLM with the text
- Execute the text to speech step.

Executing these three steps add latency. While using a multimodal LLM reduces it, in particular the end-to-end case where audio is fed to the LLM and audio is produced by the LLM.

9.6 RAG vs Long Context

Is RAG dead, since that now models can process up to 2M tokens, e.g 3000 pages. What are the benefits and trade offs of using large context windows versus RAG systems? Example personal writing assistant, or AI financial assistant?

- What are long-context models? Currently models allow 128k tokens, but Gemini Pro allows 2M tokens. Claude 3.5 Sonnet allows 200k.
- What are the benefits of large context models?
    - For example, it is possible to feed an entire codebase into the LLM
    - It is good in case processing time is not a concern: processing 2M tokens take a lot of time.
    - It is suitable for one-off tasks or smaller datasets.
- What are the benefits or RAG?
    - Lower costs than large context models (in LCLM processing 2M tokens has a greater cost).
    - It can handle vast data that cannot fit within a single LLM context window.
    - Well made RAG systems are fast and accurate.
    - Efficient document indexing ensures fast query processing.
    - RAG is a faster search than sending all data to the LLM at once
    - Relevant information can be selectively added to initial LLM prompt
    - RAG adds only relevant information to the prompt, reducing allucinations and noise.
    - RAG can be enhanced with other techniques like:
        - Metadata filtering
        - Graphs
        - Hybrid search
- What are their drawbacks?

Long context window can be used for multi document evaluation and research, where latency is not critical..

9.7 Long Context LLM(s) & Context Caching vs RAG

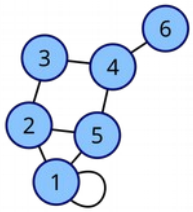
Long Context Language Models= LCLM(s)
Check Google DeepMind’s Michelangelo
Michelangelo is a new way to test how well LLMs can understand and process long pieces of information. Instead of just finding a single fact, Michelangelo challenges models to chisel away unnecessary information and focus on the important details hidden in the context. This is done using the Latent Structure Queries (LSQ) framework, which creates tasks where models must uncover a hidden structure by ignoring irrelevant data.
LCLM= Higher inteference costs and slower responses; LCLM struggle in following instructions and reasoning.
RAG has its challenges. The embedding model used in cosine similarity to find the best chunks is much less “intelligent” than the embedding space representation of the data within an LLM - the embedding scores may not encapsulate the semantic and contextual relevance of a piece of information to the query. The multi-step nature of RAG systems also introduces the risk of cascading errors, where mistakes in the retrieval phase can negatively impact the generation phase. Furthermore, RAG may face difficulties with tasks that demand complex reasoning or the integration of multiple pieces of information. The final output’s quality is closely tied to the precision of the retrieval phase, requiring careful maintenance of both the retrieval and generation components.

9.7.1 Google DeepMind's Michelangelo: Long Context Evaluations Beyond HayStacks via Latent Structure Queries

Michelangelo is a way to test how well LLM(s) understand and process long lieces of information.
Latent Structure Queries (LSQ):

9.8 Graph Machine Learning

Graphs consist of two main components: nodes (also called vertices) and edges (also called links). Nodes represent entities, and edges represent relationships between these entities. Graphs can be directed or undirected, depending on whether the relationships between nodes are one-way or bidirectional.
The unique structure of graphs presents challenges for machine learning models, which typically work on data represented as sequences or grids, such as text or images. Unlike these forms of data, graphs are unordered structures, meaning their representation must respect the fact that shuffling the order of nodes or edges does not change the graph itself. This property, known as permutation invariance, is a key consideration when designing machine learning algorithms for graphs.

An adjacency matrix is a square matrix used to represent a graph. Each entry A[i][j] is 1 if there is an edge between vertex i and vertex j and 0 otherwise. It can represent both directed and undirected graphs, with additional values for weighted graphs representing the edge weights.		$\begin{pmatrix} 2 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$

9.8.1 Graph Neural Networks (GNN(s))

Traditional GNNs face certain limitations, particularly in terms of scalability, generalization, and the challenge of over-smoothing, where deep GNN layers cause node embeddings to become indistinguishable.

9.8.2 Graph Transformer-Based Model

Transformed based models for graphs have been proposed:

- Graph-Bert
- GROVER

Despite their potential, transformer-based models for graphs also face challenges, particularly in terms of computational complexity.

9.8.3 GraphRAG

https://github.com/microsoft/graphrag

- Indexing: the input corpus is sliced into a series of chunks (text units), which serve as analyzable units throughout the process and provide detailed references in the outputs. Following this, all entities, types of entities, descriptions of the entities, relationships, and key claims are extracted from the chunks using an LLM. Then, hierarchical clustering of the graph is performed using the Leiden technique, which organizes entities into communities based on their relationships and significance. Summaries of each community and its constituents are generated from the bottom up, aiding in a comprehensive understanding of the dataset.
- Query

(The Leiden algorithm is a community detection algorithm developed by Traag et al at Leiden University)
GraphRAG is designed to address complex information discovery and analysis tasks. It excels in scenarios where the required insights span multiple documents, are obscured by noise, or are intertwined with misinformation. This tool is particularly useful when users seek more abstract or thematic answers, requiring deeper analysis beyond surface-level data.
GraphRAG's effectiveness depends on well-constructed indexing, which is crucial for applications that involve general topics or unique datasets.

10 Part 1: Building and Deploying the Final RAG Chatbot

10.1 Building the final AI Tutor (link)

Iterative and modular LLM pipeline development: the pipeline must be modular, it must be possible to swap building blocks. Integrate advanced techniques and customization. Much of becoming a great LLM developer and building a great LLM product is about integrating advanced techniques and customisation towards a specific application that can help to progress an LLM pipeline along the march of the 9s. There are many aspects to consider to improve the product before wide release:

- improving the dataset
- better prompt engineering
- advanced RAG techniques: cosine similarity, re-ranking (the LLM rankes the chunks retrieved from the Vector DB)
- Improve the UI

Create a metric to evaluate the retrieval system.
To measure the RAG retrieval effectiveness, the **metrics** are (link to the lesson) :

- Hit rate (see 4.3.1.1)
- Mean Reciprocal Rank (MRR) link

We must keep track of this metrics when evaluating optimizations.

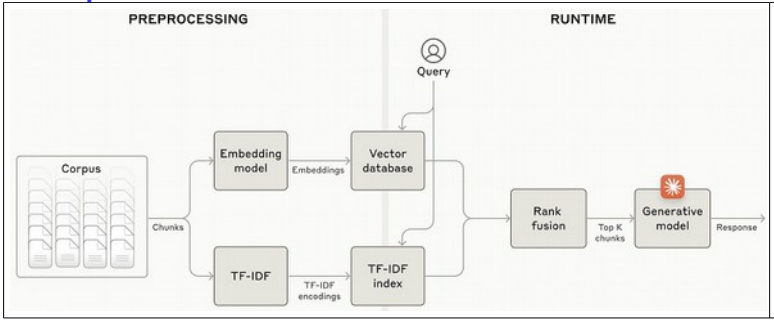
Evaluation dataset:
Generating Test Questions: we must generate question-document pairs, using an LLM. (e.g.7919 pairs)
Optimize for cost and speed.
Establish a Baseline evaluation: establish a baseline performance before introducing optimizations. The HR and MRR are evaluated with different corpuses.
Improvements:

- modify the number of retrieved chunks and evaluate the HR and MRR
- different embedding model
- Reranking:
  - Using a service by API
- Chunk size and overlap (these hyperparameters are important). The author then selected chunk size 800, overlap 0.
- Experiment with hybrid search integration (keyword search and vector DB)
- Contextual retrieval.

Contextual retrieval: a technique that can improve RAG by preserving document context during chunking. It uses an LLM to generate a brief contextual summary for each chunk, which is prepended before embedding and indexing. (link to the article)
Prompt to create context of a chunk: for each chunk, the prompt contains both the whole document and the chunk:

```
<document>
{{WHOLE_DOCUMENT}}
</document>
Here is the chunk we want to situate within the whole document
<chunk>
{{CHUNK_CONTENT}}
</chunk>
Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.
```

Anthropic Standard RAG

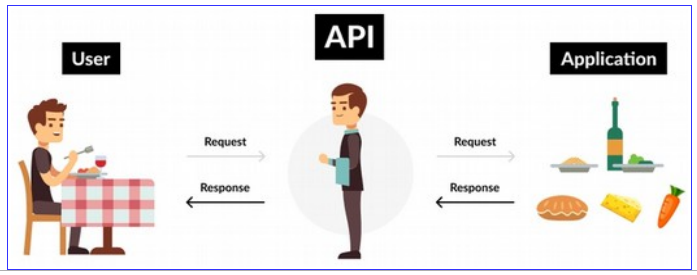


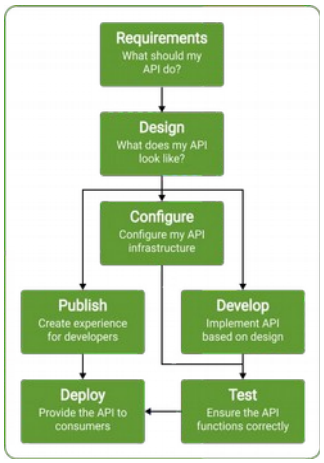
Given a query, uses an hybrid search where the corpus is searched with both:
- the embedding vector by cosine similarity
- the TF-IDF with BM25

Real user feedback and Custom Evaluations based on the Industry niche.
- Collect user questions and feedback
- use custom evaluations. For example: craft questions in which you know the answer

Continuous Improvement Strategy
ddd

10.2 How to make an API?





An API is a bridge to connect two SW applications
- Define your goals: what the API should do? How to keep it secure?
- Design: OpenAPI. Establish design rules: ensure consistency.
- Implementation
- Test
- Secure: test for vulnerability.
- Deployment: serverless / server-based deployment . Serverless has cold start-up letency.
- Observe after deployment. Monitor API performence in real time
- Distribute: make the API discoverable via internal or public catalogs.
- Manage Updates.

An API can act as a gateway, forwarding requests to different parts of the backend, loke billing, user authentication notifications, delivery, etc.
If your app involves multiple platforms, external services, or scalable services, an API is a must.
Third party API(s): access other services using their API. It is important to learn the habit of investigating what features or datasets can be accessed by third-party APIs rather than building yourself.

10.2.1 Open API

- Server generator: https://pypi.org/project/openapi-python-generator/ https://marcomuellner.github.io/openapi-python-generator/
- Python Client generator from Open API: https://pypi.org/project/openapi-python-client/
- https://swagger.io/
- Fast API can generate the Open API schema from the Pydantic Python

Uvicorn: https://www.uvicorn.org/ asynchronous application server in Python

10.2.2 HTTP Methods

GET A GET request is used to retrieve data from the server. For example, you might want to get the predictions made by your machine learning model.
POST. A POST request is used to send data to the server to create a new resource. For example, you might want to submit new data to get a prediction from your model.
PUT: Update an existing resource.
DELETE: delete an existing resource

10.3 Building and deploying a Gradio UI on Hugging Face Spaces

10.4 Quick Tips for Improving LLM(s) once in production

- **User feedback integration**: One primary task post-deployment is integrating user feedback into the LLMs. This involves collecting feedback through surveys and in-app feedback forms, as well as analyzing user behavior to understand how the model behaves in real-world scenarios
- Performance Monitoring:

- Updating to the Latest LLMs and Latest Techniques and Using the Latest Capabilities: The habit of updating to the latest, better, and more cost-effective models is fundamental for staying current in the fast-moving LLM space. Updating to the latest LLMs and new techniques also requires a robust process for testing and evaluation.
- Deploy A/B testing: comparing different models versions of configurations. Collect and analyze performance data to determine the best-performing version before updating the main deployment.
-

11 Part2: Expanding our AI Tutor

11.1 Part 2 Introduction: More Tools, Models & Frameworks for LLM Projects

11.2 LLM Agents: LLM Pipelines and Tool Use for Complex tasks

Tools that an agent can use:
- Search engines
- Language translation API(s)
- Image generation
- Database query tools

The agent interacts with: tools, memory, planning module.
Planning can happen:
- without feedback
- with feedback

The agent operates on a think, act, observe loop.
- Think: analyze the current situation and plan the next move
- Act: use a tool or provide the final answer

12 Part 2: More LLM Frameworks and Tools

12.1 More LLM framework options

Compares LlamaIndex, LangChain and OpenAI Assistants. LlamaIndex is more focused on RAG, LangChain aims to be a bit more general.
- LangChain: modular and flexible
- LlamIndex: processing, structuring and accessing private or domain specific data.
- OpenAI Assistants: Assistant API – Saas

12.2 LangChain introduction 101

LangChain is an ecosystem composed of:
- LangChain
- LangGraph:
- LangServe: deployment of LangChain chains as Rest API(s): the site recommend to use LangGraph for new products.
- LangSmith: agents observability

12.2.1 LangChain concepts and Components

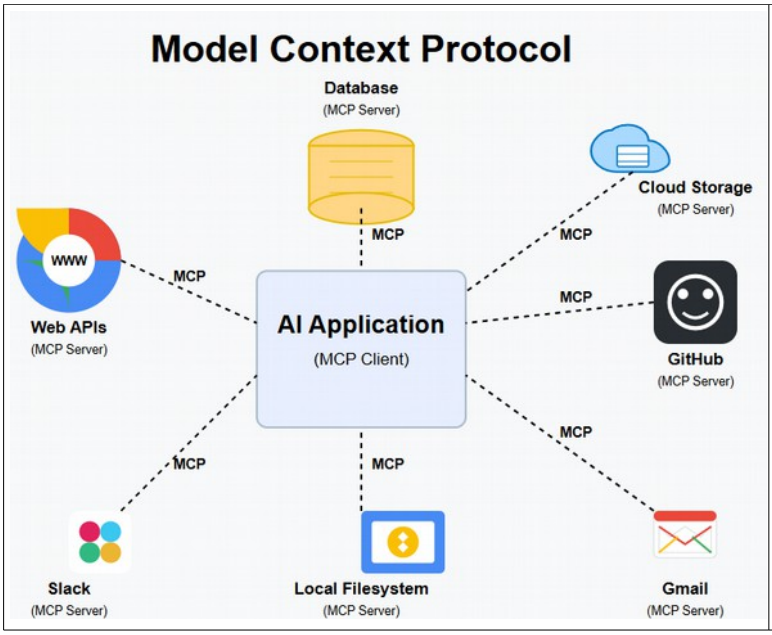
12.3 Access More AI Models with Hugging Face's API(s)

12.4 Extending LLM Capabilities with Anthropic MCP

12.4.1 Why MCP? Integration bottleneck

- MCP server allows to access a tool or data source.
- The application using an LLM is called MCP Host (like in USB), it uses the MCP Client library to discover and interact with any MCP server.

12.4.2 MCP Architecture



MCP Client within the host application. Each MCP Client interacts with one specific MCP Server.
The developer building the MCP client do not interact directly with the client.
MCP has standardized primitives for:

- **Resources**: data that the client or LLM can read. Files, database content,etc. Each resource has an URI. Resources are application controlled: the host application decides which resources are accessed.
- **Tools**: actions or functions. The model usually decides which tool to access.
- Prompts: pre-defined text templates or structured interaction flows that the server can offer.

12.5 Observability and Tracing in Production

LangSmith cannot used in self hosted mode when free.
https://phoenix.arize.com/ Phoenix Arize can be used locally.
https://langfuse.com/ Lang Fixe can be used locally

12.6 More API(s) and Tools

- Zapier (https://zapier.com/): workflow automation and agents.
- Crew AI.
- LangChain integrates:
  - DuckDuckGo search https://python.langchain.com/docs/integrations/tools/ddg/
  - Brave Search is Free: https://python.langchain.com/docs/integrations/tools/brave_search/
  - Google Serper https://python.langchain.com/docs/integrations/tools/google_serper/

13 Part 3: LLM Ecosystem Economics; Choose the Right Niche and Business Strategy for your LLM Pipeline (9 Lessons)

13.1 What Will Drive Enterprise Adoption of Generative AI?

- The high bar for reliability: hallucinations are unacceptable in critical business functions. Create robust solutions using RAG, fine tuning, prompt engineering, agentic workflows, iterative improvement and evaluation. Product idea insight: when developing an LLM product, focus on areas where even incremental reliability improvements can deliver value
- Customization and contest. Generic LLM(s) lack the specific internal context. Focus on Data curation and preparation, custom RAG pipelines, fine tuning for specialization, UI/UX considerations. Look for enterprise workflows reliant on internal knowledge.
- Security and Data Privacy Concerns
- Demonstrating ROI
- Talent Gap and Organizational Inertia

13.2 Who are the Generative AI winners?

Value chain:

- AI HW and GPU designers:

- AI cloud platforms
- Closed LLM models
- Open weights models
- Open source platforms to access models (Hugging Face)
- Access to LLM via consumer products : ChatGPT, perplexity, bing
- Individual gen AI product builders: startups, side-projects, personal tools.
- Enterprise in-house LLM pipeline development for aiding internal workflows or improving external products
- Tools for AI and LLM developers (LangChain, Llamaindex, etc.)

If LLM pricing is a race to the bottom, the biggest winners are the people and companies that use them to create value for themselves. *It is difficult to measure the return on investment on GPU spending when many of the gains may be productivity and cost savings rather than standalone new AI revenue*.

13.3 Impact of Large Language Models on Different Job Categories

Key Themes:
- Impact of LLMs on Jobs: LLMs predominantly assist with routine and repetitive tasks; creative and specialized tasks still rely heavily on human input. Collaborative human-AI workflows will be the norm.
- LLM Evolution Stages:
  1. Unreliable Intern Assistants: Basic help with significant human oversight.
  2. Co-pilots: Productivity aids for human-led work.
  3. Agents: More complex autonomous task management (with supervision).
  4. Fully Automated Workflows: Future goal, task-dependent.

Sector-Wise Impact:
1. Knowledge Work:
   - Enhanced efficiency in research, data analysis, content creation, and customer service.
2. Creative & Artistic Work:
   - Assistance in writing, design, and music composition. Human creativity and vision remain critical.
3. Technical & Specialized Work:
   - Augmentation in programming, legal tasks, financial analysis, insurance, and medical diagnostics. Humans oversee complex problem-solving and final decisions.
4. Physical Labor:
   - Indirect benefits via optimization and robotic integration. Limited direct impact for now.
5. Management & Leadership:
   - Decision support, project management, and performance analysis.
6. Education:
   - Personalized learning, tutoring, and content creation. Human educators vital for socio-emotional growth.
7. Routine Tasks:
   - High susceptibility to automation (e.g., data entry and administrative jobs).

Emerging Opportunities:
- AI-related roles, such as LLM Developers, Prompt Engineers, and Ethics Specialists, are on the rise.

13.4 175 Project Ideas

Questions:
- Primary function
- Target User
- Industry and Domain
- Value Proposition and Defensibility

13.4.1 Technology and SW

- API design and documentation co-pilot
- automated SW architecture diagram and system explainer

13.5 What Makes a Defensible Product Built With Foundation LLMs?

What should we build ourselves, and where should we use third-party AI tools? A defensible generative AI product must address an industry niche or workflow where you:
- Have a very specific internal workflow or product where a customized LLM pipeline will get improved results or
- You already have or have a plan to develop a data or industry expertise advantage.

Pick a niche and bring in human domain expertise Generic tasks will be solved by foundation models, products built by a leading AI lab, or by the open source community. To compete and build something defensible you need: data, expertise and users that OpenAI and Google do not have.

Leverage Proprietary and domain-specific data Example, in an healthcare project, use anonymized data and provide secure access via RAG. The resulting model can be more accurate and more relevant to the population served by the product.

Go for First-mover advantage with innovation in application and use cases. The most defensible products are often those that create entirely new categories. Think beyond incremental improvements or replicating other apps. Instead, imagine entirely new ways of solving problems with AI and get a first-mover advantage. If you are early in tackling a new class of problems, you can get early user adoption and revenues to fund growth or justify the internal budget; otherwise, you may be able to raise investment from venture capital to become an AI leader.

Build Customization with Prompting, RAG, Fine-Tuning, and Agents it rarely makes sense to train your own custom LLM. However, it's not enough to simply implement an API call to GPT-4 or another advanced model; true differentiation comes from

the layers built around the foundation model. The key is to identify your unique strengths or those of your team and organization and leverage these to create a product that's more than the sum of its parts. The art and science of data set curation, prompt engineering, RAG, agents, and fine-tuning are often underestimated.

Human-Centered AI Integration Is the Key to User Adoption Understand your target users, address their issues, and build a highly customized UI/UX that allows them to easily adopt it into existing workflows. In particular, make it easy for users to understand the model's weaknesses and efficiently identify and correct its mistakes.

Leverage The Flywheel Effect with Continuous Learning and Improvement A key power of AI lies in its ability to improve over time. However, designing effective learning loops is challenging. It requires careful consideration of data quality, bias mitigation, and performance metrics. Continuously evaluate your model and continue the march of 9s to cover more edge cases and achieve improved accuracy and performance. Track failures or suboptimal LLM responses and use these to learn. Try to gather success signals from your users that you could use for in-context learning examples or fine-tuning datasets to improve your model further.

Build a Moat With Ecosystem Integration and Network Effects From an economic perspective, network effects are one of the strongest drivers of defensibility in tech products. With LLM-based products, these effects can be even more pronounced due to the data-driven nature of AI. We've seen great success in platforms that allow third-party developers to build on top of their AI capabilities. This expands the product's utility and creates a community of stakeholders invested in its success. The more developers build on the platform, the more valuable it becomes, creating a powerful moat against competition.

Addressing Ethical Considerations and Building Trust Can Be a Competitive Advantage: invest in developing explainable AI techniques that allow you to provide transparency in your model's decision-making process.

Make Regulatory Compliance and Security an Opportunity

Don't Forget Cost and Resource Management

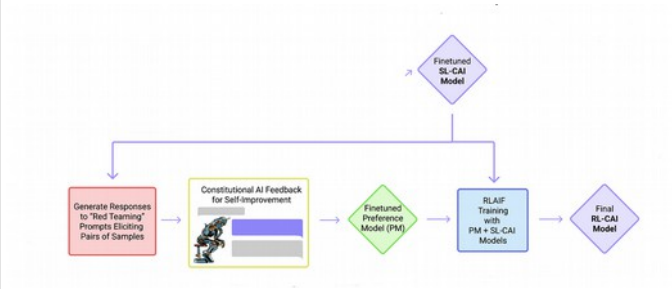
14 Glossary and Terms

Term	Description
RLAIF	Reinforcement Learning on AI Feedback: similar to RLHF, but here the AI Teacher can be a bigger model, or an ad-hoc model that evaluates the response of the LLM and provides reward. It is unsupervised
RLHF	Reinforcement Learning on Human Feedback. The Human responses are used to train a model that evaluates the model response and gives an reward.

15 Papers

https://arxiv.org/abs/2408.03314v1

15.1 Constitutional AI: Harmlessness from AI Feedback (link)



Human input from a set of principles that govern the AI behavior along with a small set of examples: they form the constitution.
Supervised stage: Critique → Revision → Supervised learning:

- generate harmful responses
- for each of them, draw a random contitutional principle, and ask the model to critique its response based on the constitutional principle
- so collect a set of actual responses and corrected responses.
- Fine tune the model with the pairs

RL stage:RLAIF, the AI evaluates it s responses according to the Contitutional principles.

16 Moreover

16.1 Model Context Protocol (MCP)

As of today, use Claude Desktop.
An MCP server exposes tools.
MCP host: an application that wants to access data via MCP: resources exposed via can be data or applications exposed via MCP. Example of MCP hosts are Cursor and Claude Desktop. The MCP client is the extenson to an application that manages the protocol connection via MCP. The MCP server (what is generally called MCP) exposes some resource via MCP.
What can MCP do?

- Resources: providing direct access to data
- Prompts: providing the language model with customized prompts (create reusable prompt templates and workflows)
- Tools: preforming actions on or with the data.

There are other capabilities in the MCP protocol:

- Sampling: set the server request completions from the LLM
- roots: clients inform servers about resources and their locations

16.1.1 Blast radius of AI agents

16.1.2 Creating an MCP server with Python

MCP server can expose:

- Resources: (client controlled) exposes data to be used as context.
- Tools (model controlled)
- Prompts (user controlled): the MCP server can expose reusable prompts and workflows to users.

Use the MCP Inspector to test and debug MCP servers.

16.1.3 Links

- https://mcp.so/ awesome MCP servers.
- https://www.pulsemcp.com/ Pulse MCP: MCP Clients: https://www.pulsemcp.com/clients

17 Biography

ID	Title	Version

18 Links

What is AI

Editable Google Doc

Commentable Google Doc

https://aitutor.towardsai.net/

Course GitHub repo with Jupyter notebooks
https://jobs.towardsai.net/
Discord Channel : https://discord.gg/learnaitogether the channel is: #beginner-to-advanced-llm-developer .
https://www.louisbouchard.ai/

19 Tools

19.1 AI Based Search Engines

- https://consensus.app/ Consensus.

19.2 Companies

- Gradient : a company that provides fine tuning

19.3 Deployment

- TorchChat run LLM(s) locally using Python, C++, on Android or iOS.

19.4 Development Tools

- https://www.firecrawl.dev/
- dotenv is a Python package

19.5 Models

- Mistral AI documentation.

19.6 Text To Speech

- https://smallest.ai/text-to-speech/spanish
- https://elevenlabs.io/pricing

19.7 Training

19.7.1 Prompt Engineering

- Learn Prompting a site about prompt engineering

RAG
https://learn.activeloop.ai/courses/llms

19.7.2 Post-Training

- Torchtune:a PyTorch library for easily authoring, post-training, and experimenting with LLMs.

20 Terms and acronyms

Term	Description
Hallucination	The model returns a response that seems true, but it is not.
logits	the vector of raw (non-normalized) predictions that a classification model generates, which is ordinarily then passed to a normalization function. If the model is solving a multi-class classification problem, logits typically become an input to the softmax function. The softmax function then generates a vector of (normalized) probabilities with one value for each possible class.